

基于序贯概率比下的生产决策最优规划模型

摘要

本文通过序贯概率比检测, 0-1 线性规划模型和动态规划等方法解决了生产企业在生产过程中的决策优化问题。

针对问题一, 本文设计了一种基于序贯概率比检验 (SPRT) 的抽样检测模型, 根据 95% 置信度的情况下拒收, 90% 置信度情况下接受的题目条件, 得到显著水平、功效, 基于二项分布构造似然比, 随着抽样检测数量不断累积, 次品率也不断累加, 达到边界输出结果值, 结果显示当抽样检验达到 263 个时, 有 90% 的信度认为次品率不超过标称值, 接受零件; 当抽样检测达到 384, 在 95% 信度下认为次品率超过标称值, 拒绝这批零件。最后用 oc 曲线检验得到模型能有效拒绝高次品率方案的结论。

针对问题二, 通过决策树做出生产流程图, 采用了 0-1 规划和单目标规划方法, 针对企业生产过程中检测、装配、拆解和退换策略, 建立了生产成本优化模型。通过遍历所有零件和成品检测与否的决策组合, 计算各方案下的生产成本、收入、正品数量、装配数量等, 确保在不同次品率条件下实现总利润最大的投放。得到对于情况 1, 情况 2 和情况 3 最优策略是检测零件 1 和零件 2, 成品既不检测也不拆解; 对于情况四, 零件 1, 零件 2 和成本均进行检验, 成品进行拆解。情况 5 最优策略是对于零件 1 和零件 2 不进行检验, 成品进行检验但不拆解; 情况 6 最优策略是对零件 1, 零件 2 和成本均不进行检验, 成品也不进行拆解。

针对问题三, 在问题二的基础上, 将零件与零件合成半成品的过程引入模型, 并计算出在不同决策条件下的半成品次品率。我们将半成品视为新的零件, 将其次品率作为零件次品率, 进而在半成品合成成品的过程中, 应用与问题二相同的决策优化方法。通过遍历所有可能的决策组合, 对零件、半成品和成品的次品率进行综合分析, 并借助动态规划算法, 最终得出成本最小化和正品率最大化的最优生产方案。最优方案为零件 1 到 8 均进行检验, 半成品都不进行检验和拆解, 成品进行检验并拆解。

针对问题四, 由于序列方法的样本量能影响次品率, 首先按照题目要求对问题二和问题三, 以样本量为自变量, 最大利润为目标函数建立总模型。数目规模不同的决策为: 对于小规模, 如果次品率较低 (例如 $< 10\%$), 则不对零配件 1 和零配件 2 进行检测, 如果次品率较高, 则对成本较高的零配件进行检测; 对于中等规模, 如果次品率低于 10%, 可以减少检测零配件的频率, 只检测其中一个; 对于大规模, 如果次品率超过 10%, 对两个零配件进行检测, 并对成品进行随机抽样检测; 当生产规模较大时, 如果次品率估计较高 (例如超过 15%), 全面检测零配件 1 和零配件 2, 同时也对成品进行全面检测。

最后, 本文对问题一模型和问题三模型进行了检验, 验证了模型的可行性。

关键词: 序贯概率比检验 动态决策模型 生产过程决策 决策树

目录

一、 问题背景与重述	3
1.1 问题背景	3
1.2 问题重述	3
二、 问题分析	3
2.1 问题一分析	3
2.2 问题二分析	4
2.3 问题三分析	4
2.4 问题四分析	5
三、 模型假设与符号说明	5
3.1 模型假设	5
3.2 符号说明	6
四、 模型的建立与求解	6
4.1 问题一模型的建立与求解	6
4.2 问题二模型的建立与求解	11
4.3 问题三模型的建立与求解	15
4.4 问题四模型的建立与求解	18
五、 模型评价	20
六、 模型检验	20
七、 参考文献	24
八、 附录	24
A 附录一：问题一代码	24
B 附录二：问题二代码及结果	27
C 附录三：问题三代码	32
D 附录四：问题三模型检验 matlab 代码	34

一、问题背景与重述

1.1 问题背景

一家电子厂面临在原料购买，生产，销售三个方面的问题。在生产和销售阶段，生产成品（半成品）由半成品（零件）进行组装获得。通过拆分决策和检验决策能很好地降低产品的生产成本，提升平均产品质量（次品率降低），从而提高公厂的声誉。在购买原料阶段，因为无法对原料进行完全检验，所以有的次品原料会进入生产过程。优秀的抽样方法可以更好地估计总体，对次品率有更清楚地估计，使得最优策略的确立更明确，给工厂带来最大化收益。题目聚焦于帮助企业解决以下核心问题：一是设计高效经济的抽样检测方案，以判断供应商提供的零配件次品率是否超标，为接收或拒收决策提供科学依据；二是基于已知的零配件及成品次品率，优化生产流程中的检测、拆解及退换策略，以最大化利润并最大化市场投放的合格产品量。具体而言，包括决定何时检测零配件和成品、如何处理不合格品，以及面对用户退换请求时的应对措施。通过这些决策，企业能够在保障产品质量的同时，有效控制成本，提升市场竞争力。

1.2 问题重述

1. 设计抽样检测方案，以最少检测次数验证供应商零配件次品率是否超过标称值(10%)，分别在 95% 和 90% 的信度下决定是否接收或拒收该批产品。
2. 基于已知次品率，为生产过程各阶段制定检测、拆解和退换决策，优化成本与质量。
3. 对 一道工序、一个零配件，已知零配件、半成品和成品的次品率，重复问题 2，给出生产过程的决策方案。图 1 给出了 2 道工序、8 个零配件的情况，具体数值由表 2 给出。
4. 基于问题一抽样方法抽样检测得到的次品率估计值，重新优化问题二和问题三中的决策方案，以应对次品率的不确定性。

二、问题分析

2.1 问题一分析

在购买原料阶段，次品率是造成损失主要来源，工厂和供应商达成协议，在双方都能接受的置信区间内，该批零件不会超过规定的标称值，如若违背工厂拒收。现要求如果标称值为 10%，根据抽样检测方案，针对以下两种情形，分别给出具体结果：

- (1) 在 95% 的信度下认定零配件次品率超过标称值，则拒收这批零配件；
- (2) 在 90% 的信度下认定零配件次品率不超过标称值，则接收这批零配件。

设计精确，经济的抽样检测方法，精确地估计了零件的次品率，使工厂清楚地做出最优策略，获得最大收益。对于这种抽样方案的设计，通常采用传统的计数抽样，但是传统的计数型抽样检验通常假定不合格品率为固定常数，且不存在检验错误。实际生产中，不合格品率往往未知且为随机变量，检验错误也难以避免。为降低抽样方

案造成的风险及损失，可利用动态，多阶段的序贯概率检验。这种方法能大幅度地减小平均试验样本量与平均试验时间，从而有效地降低产品抽样检验的试验成本^[3]。序贯概率检验 (Sequential Probability Ratio Test, SPRT) 是一种在数据逐步获取的过程中进行决策的统计方法，它允许我们在获取数据的过程中逐步更新决策，而不需要事先确定固定的样本量。这意味着我们可以根据实际情况动态调整数据获取量，从而在保证决策准确性的同时降低数据收集成本。

由于序贯概率比检测涉及到可调整的偏差值和样品量上限，我们还需要对它们进行调整，求解出检测最稳定的偏差值和样品量上限，进一步完善检测模型。最后考虑引入一些方法检验模型的可行性。

2.2 问题二分析

问题二，工厂的装配流程包括两个主要决策，检验决策和检验过后发现是不合格产品的拆解或者丢弃决策。

(1) 对零配件是否进行检测，如果对某种零配件不检测，这种零配件将直接进入装配环节；否则将检测出的不合格零配件丢弃；

(2) 对装配好的每一件成品是否进行检测，如果不检测，装配后的成品直接进入市场；否则只有检测合格的成品进入到市场；

(3) 对检测出的不合格成品是否进行拆解，如果不拆解，直接将不合格成品丢弃；否则对拆解后的零配件，重复步骤 (1) 和步骤 (2)；

(4) 对用户购买的不合格品，企业将无条件予以调换，并产生一定的调换损失（如物流成本、企业信誉等）。对退回的不合格品，重复步骤 (3)。

现要求已知两种零件（A、B），表一中，所有的次品率，购买成本，检测成本，购买单价，检测成本，市场售价，调换和拆解费用都是已知的，给出具体的决策。

这是一个优化问题，旨在寻找到各个情况下成本最低的决策方案。成本包括采购成本、检测成本、装配成本、调换损失和拆解费用。本文首先考虑不计拆解费时的成本，此时目标函数只与是否检测有关。检测分为零件 1 检测、零件 2 检测和成品检测，所以 6 种情况下产品的生产只有 2^3 种路径（不同的路径成本大概率不相同），考虑算出 8 种路径的成本，再相互比较，确定检验方案。而后再对是否拆解做出分析与判断。

2.3 问题三分析

问题三要求在第二问的基础上，对 2 道工序、8 个零配件的生产过程进行决策，最终在已知零配件、半成品和成品的次品率的情况下，为 m 道工序和 n 个零配件的生产过程制定优化决策方案。

问题将生产的复杂性扩展到了多道工序和多个零配件的情况。每道工序都有相应数量的零配件，这些零配件组装成若干半成品，最后进行成品的组装。需要优化的决策方案涉及以下几个方面：

1. 是否对每个零配件进行检测；检测后不合格品是否丢弃。
2. 是否对每个半成品进行检测；检测后不合格品是否拆解并重复检测。

3. 是否对最终成品进行检测；不合格成品是否拆解并重新利用。

优化的目标函数还是最小总成本。为了简化计算，我们暂时忽略成品和半成品的拆解费用，只关注零件和半成品在装配和检测过程中的成本。由此，对于总成本的计算，我们需要计算出零配件成本、半成品装配成本、半成品和成品检测成本、市场损失和替换成本。我们大体可以按照四步进行计算：

第一步是计算八个零件的成本和检测成本。这些成本是所有生产步骤的基础，因为每个零件的成本直接影响到生产总成本。

第二步是计算零件装配成半成品的费用。由于题目中给出的数据表明八个零件装配成三种半成品的装配费用都是相同的（每个装配费用为 8 元），因此可以将不同的半成品装配费用一同计算，这样可以简化公式和计算过程。

第三步，分步骤计算三个半成品的次品率。半成品的次品率由三种情况构成：

(1) 正品和正品装配后的半成品次品：表示两个都是正品的零件在装配成半成品时可能产生的次品率。

(2) 正品和次品装配成的半成品次品：表示一个正品零件和一个次品零件装配成半成品时产生的次品率。

(3) 次品和次品装配形成的半成品次品：表示两个次品零件装配成半成品时产生的次品率。

第四步是将半成品当作新的零件，然后重复上述步骤来计算它们在进一步装配和检测过程中的成本和次品率。

经过计算出半成品的次品率，我们可以算出成品的次品率，从而算出成品检测费用和调度损失，最终算出总成本。

2.4 问题四分析

假设问题 2 和问题 3 中零配件、半成品和成品的次品率均是通过序贯概率比检测方法抽样检测得到的。

(1) 利用 SPRT 对零配件 1、零配件 2 和成品进行抽样检测，动态估计其次品率，然后更新到更新后的次品率 P_{new} 。

(2) 将更新后的次品率带入问题二和问题三的优化模型。更新模型的成品率，单件物品成本，正品数量，最大利润。从而得到最优策略。

(3) 由于 SPRT 对检测样本总数比较敏感，我们特别对不同的样本总数进行策略的优化。

三、模型假设与符号说明

3.1 模型假设

1. 假设生产的产品都能被售卖掉，不考虑储存成本。
2. 假设表 1、表 2 的数据合理有效，无异常值。
3. 假设顾客在拿到不合格产品后一定会进行调换。
4. 假设在一个顾客多次调换的情况下单次调换产品的调换损失费用恒定不变。

5. 假设不同零配件（零配件 1、零配件 2 等）之间的次品率是相互独立的，同一批次内，各零配件或成品的次品情况也是相互独立的。
6. 假设检测过程能够准确识别出不合格的零配件或成品，且检测本身不会对零配件或成品造成任何物理损坏。
7. 假设拆解过程同样不会对零配件造成损坏，拆解后的合格零配件可重新使用。
8. 假设成品拆解之后直接变成零件。

3.2 符号说明

表 1 符号说明

符号	说明	单位
α	显著性水平	
β	功效	
p_0	原假设下的次品率	
p_1	备择假设下的次品率	
λ_n	似然比	
k	检测到的次品数	件
n	样本总数	件
δ	超出的次品率，即偏差值	
C_1	零配件 1 的购买单价	元
C_2	零配件 2 的购买单价	元
d_1	零配件 1 的次品率	
d_2	零配件 2 的次品率	
C_{t1}	零配件 1 的检测成本	元/件
C_{t2}	零配件 2 的检测成本	元/件
$C_{\text{装}}$	成品装配成本	元/件
d_3	成品合成的次品率	
d_t	成品的次品率	
C_{t3}	成品检测成本	元/件
$C_{\text{市}}$	市场售价	元/件
$C_{\text{调}}$	调频损失	元/件
$C_{\text{拆}}$	拆解费用	元/件

四、模型的建立与求解

4.1 问题一模型的建立与求解

Wald 提出了序贯概率比检验的算法，其基本原理为：设 $x_1, x_2, \dots$ 为一组满足独立同分布的随机变量序列^[10]。每个 x_i 表示样本群 $\{x_i\}$ 的一个观测数据，当 $\{x_i\}$ 为离散型随机变量时，其条件概率分布可表示为 $f(x/\theta)$ ，样本群 $\{x_i\}$ 的分布依赖参数 θ 。

对于一个二元序贯概率比检验，零假设和被择假设分别表示为 $H_0 : \theta = \theta_0$ 和

$H_1: \theta = \theta_1$ 。它们的联合分布密度函数为

$$f_j(x) = p(X_1, \dots, X_n/H_j) = \prod_{i=1}^n f(x_i/\theta_j)$$

其中, $j = 0, 1$ 。序贯概率比检验的似然比 λ 为

$$\lambda_n(x) = \lambda_n(x_1, \dots, x_n) = \frac{p(X_1, \dots, X_n/H_1)}{p(X_1, \dots, X_n/H_0)} = \frac{\prod_{i=1}^n f_1(x_i/\theta_1)}{\prod_{i=1}^n f_0(x_i/\theta_0)}$$

一. 模型的建立

本文现将序贯概率比检验^[1] 的方法应用于题目的抽样检测, 具体步骤如下:

1. 原假设的设置

首先, 我们需要定义原假设 H_0 和备择假设 H_1 , 根据题目, 标称值为 10%:

- H_0 : 次品率 p 不超过 10%, 即 $p \leq 0.10$ 。
- H_1 : 次品率 p 超过 10%, 即 $p > 0.10$ 。

为了进行统计检验, 需要根据样本数据, 计算出某些概率量 (p), 并通过这些数据来判断是否接受或拒绝 H_0 。

2. 似然比的构建

似然比 λ_n 是检验过程中用于衡量两种假设 (H_0 和 H_1) 下, 样本数据出现的概率的比值。

二项分布的引入:

在次品率检验中, 次品的出现服从二项分布 (即次品与非次品)。设某次检测的样本量为 n , 次品数为 k 。我们可以假设次品率 p 是已知的, 那么在 H_0 和 H_1 下, 次品的概率分别为:

- p_0 : 原假设下的次品率 (设为 0.10)。
- p_1 : 备择假设下的次品率, 可以设为 $p_1 = p_0 + \delta$, 其中 δ 是超出的次品率。

二项分布下, 样本中 k 个次品的概率为:

$$P(k | p, n) = \binom{n}{k} p^k (1-p)^{n-k}$$

根据这个公式, 分别计算 H_0 和 H_1 下的次品数出现的概率, 并构造似然比 λ_n :

$$\lambda_n = \frac{P(k | p_1, n)}{P(k | p_0, n)} = \frac{\binom{n}{k} p_1^k (1-p_1)^{n-k}}{\binom{n}{k} p_0^k (1-p_0)^{n-k}}$$

简化后:

$$\lambda_n = \left(\frac{p_1}{p_0}\right)^k \left(\frac{1-p_1}{1-p_0}\right)^{n-k} \quad (1)$$

其中, k 是检测到的次品数, n 是样本总数。

3. 显著性水平与功效的设定

显著性水平 α 和功效 β 是假设检验中的两个重要参数:

- **显著性水平 α :** 在 H_0 为真的情况下, 错误地拒绝 H_0 的概率, 由于题目是在 95% 的信度下认定零配件次品率超过标称值, 故取 $\alpha = 0.05$ (表示有 5% 的可能性误判)。

• **功效** $1 - \beta$: 在 H_1 为真的情况下, 正确拒绝 H_0 的概率。 β 通常表示在 H_1 为真时错误接受 H_0 的概率。

设定 $\alpha = 0.05$ 和 $\beta = 0.10$, 可以理解为:

• 当零配件次品率 p 真正为 10% 时, 我们希望 95% 的可能性拒绝批次 (也就是说允许一定误差);

• 当次品率真正超过 10% 时, 我们希望 90% 的情况下能够检测出来并正确拒绝批次。

4. 决策边界的推导

为了做出拒绝或接受的决策, SPRT 方法中定义了两个边界值 A 和 B , 分别用于拒绝或接受原假设。

根据 SPRT 的设计, 决策边界由以下公式给出:

$$A = \frac{1 - \beta}{\alpha} \quad (2)$$

$$B = \frac{\beta}{1 - \alpha} \quad (3)$$

将代入 $\alpha = 0.05$ 和 $\beta = 0.10$ 分别带入 (2) 式和 (3) 式:

$$A = \frac{1 - 0.10}{0.05} = \frac{0.90}{0.05} = 18$$
$$B = \frac{0.10}{1 - 0.05} = \frac{0.10}{0.95} \approx 0.105$$

这两个边界值是在进行检测时用于判断的关键标准:

- 如果似然比 $\lambda_n \geq A$, 则拒绝该批次零配件 (即认为次品率超过了 10%)。
- 如果 $\lambda_n \leq B$, 则接受该批次零配件 (即认为次品率不超过 10%)。
- 如果 $B < \lambda_n < A$, 则继续检测, 直到累积数据足够做出决策。

5. 决策过程

在每次检测新的样本时, 都会更新累积的似然比 Λ_n , 并根据当前的似然比与边界 A 和 B 的比较结果, 来决定是接受、拒绝还是继续检验:

- 继续检测: 若 $B < \lambda_n < A$, 则继续检测, 直到做出决策,
- 拒绝批次: 如果 $\lambda_n \geq A$, 则拒绝该批次
- 接受批次: 如果 $\lambda_n \leq B$, 则接受该批次。

6. 数值模拟的考虑

通过对每次样本进行序贯检测, 模型可以对批次的次品率进行有效监控。为了验证该方法的有效性, 可以进行数值模拟, 以计算在不同次品率下的检验功效和实际决策效果。

模拟结果预期:

• 在 $p = 0.10$ (标准次品率) 下, 应该大部分样本的似然比在上下界区间内 (即 $\lambda_n \leq B$)。

• 在 $p = p_0 + \delta$ (次品率超标) 下, 预期在较早的检测中似然比就会大于 A , 从而快速拒绝该批次。

进一步讨论

(1) 偏差值的选取

在本题的统计分析、实验设计及数据建模过程中，偏差值 δ 代表了数据中的不确定性或噪声水平，其大小直接关联到所需样本量的大小以及最终结论的稳健性。本文将对偏差值的灵敏度进行分析。

本文将偏差值取为 0.01、0.02、0.03、0.04、0.05，分别计算出在总体量为 1000、2000、5000 和 10000 时所需要选取的样本量个数：

偏差	偏差	偏差	偏差	偏差	偏差
0.005	0.01	0.015	0.02	0.025	0.03
样本数	样本数	样本数	样本数	样本数	样本数
754	468	234	130	850	560
偏差	偏差	偏差	偏差		
0.035	0.04	0.045	0.05		
样本数	样本数	样本数	样本数		
420	310	230	180		

图 1 样本上限为 10000 时结果

结果可视化：

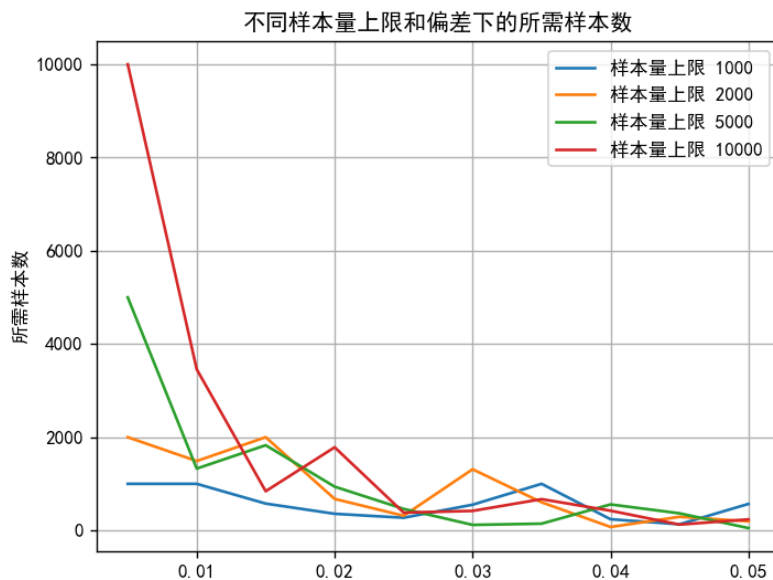


图 2 不同总体量和偏差值下所需的样品数

整体上看，对于四种样本上限，所选取的偏差值越小，需要的样本量就越大，因为当偏差较小时，样本与总体之间的差异较小。从 0 到 0.01 这段区间内，总体所需样本数随着偏差值的增加而飞速下降；在 0.01 到 0.04 在段区间内，变得波动；在 0.04 到 0.05 之间趋于稳定。

为了更准确地反映总体的特征，需要更多的样本来确保数据的稳定性和可靠性。然而，随着偏差参数的逐渐增大，样本量开始减少。当偏差增大到一定程度时，继续增加样本量对于减少偏差的边际效益递减，甚至可能因成本过高而不再经济可行。

由上图可知，当偏差值在 0.03 左右时，无论总体取值为多少，样本量都趋于平稳，于是在该题的背景下，我们选择 0.03 作为超出的次品率，即偏差值。

(2) 总体量的选择

当总体量较小时，样本量的变化对总体量的变化非常敏感。在较小的总体量下，即使总体量有微小的增加，也可能需要显著增加样本量来保持数据的准确性和可靠性。在总体量较大的情况下，样本量的增加对总体量变化的敏感度降低。这是因为在大规模总体中，样本的代表性已经相对较高，因此不需要过于频繁地调整样本量来适应总体量的变化。

二. 模型的求解

本文以偏差值为 0.03，在总体量为 10000 为例，求取当偏差值为 0.03，在总体量为 10000 时，所需的样品量个数。结果如下：

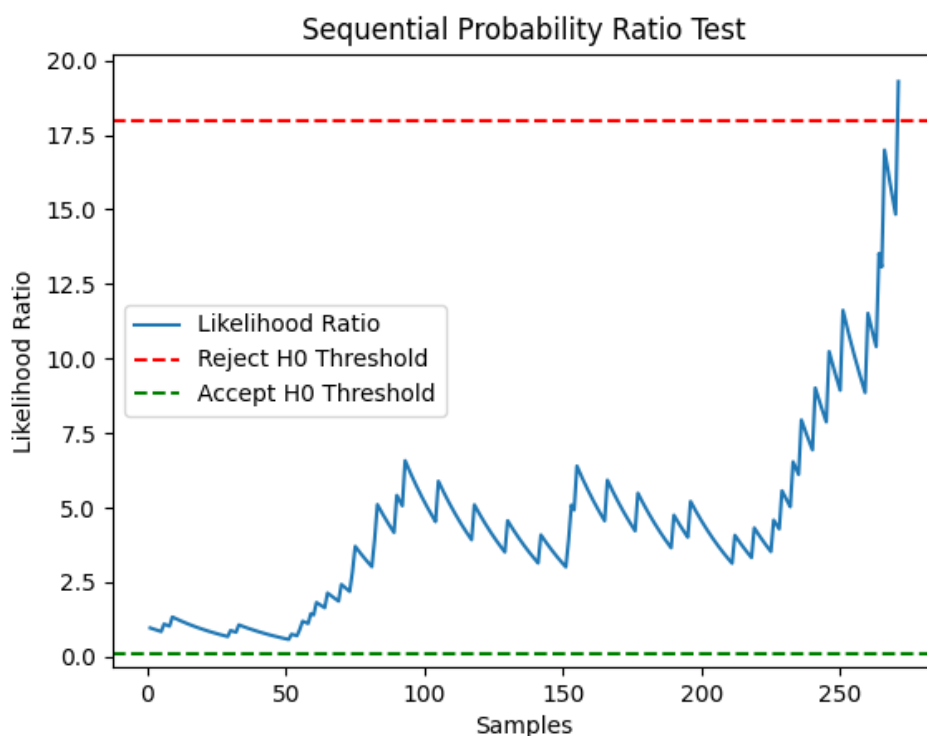


图 3 0.03 偏差值，总体量为 10000 时的序贯概率比检验

上述图像展示了使用序贯概率比检验 (Sequential Probability Ratio Test, SPRT) 方法进行统计检验的结果。

纵轴表示似然比，这是检验的核心度量，用来判断我们是否可以接受或拒绝零假设 H_0 。似然比越大，越倾向于拒绝 H_0 。横轴表示样本数量 (Samples): 随着样本数量的增加，似然比的变化被跟踪和记录下来。红色虚线 (Reject H_0 Threshold) 是拒绝零假设 H_0 的临界值。如果似然比曲线超过这条线，就表示我们有足够的证据拒绝 H_0 。绿色虚线 (Accept H_0 Threshold) 是接受零假设 H_0 的临界值。如果似然比曲线低于这条线，就表示我们有足够的证据接受 H_0 。

由图像可得：

随着样本量的增加，似然比最初在一个较小的范围内波动，这表示检验开始时还没有明显的趋势。在接近图像的中后期，似然比曲线逐渐上升，表明数据越来越偏离零假设 H 。最终，似然比在接近图像末端的时候迅速上升并超过了红色拒绝线 (Reject Threshold), 这表明根据序贯概率比检验结果，可以拒绝零假设 H_0 , 即我们有足够的

证据认为数据存在偏差。该检验在样本量达到约 260 个左右时，达到了拒绝零假设的标准。

三. 模型的检验

对于模型检验，本文根据 OC 曲线分别评估模型抽样的质量水平。

OC 曲线 (Operating Characteristic Curve):

OC 曲线，即接收概率曲线或抽样特性曲线，它直观地展示了批量产品被接收的概率如何随批次次品率的变化而变化。

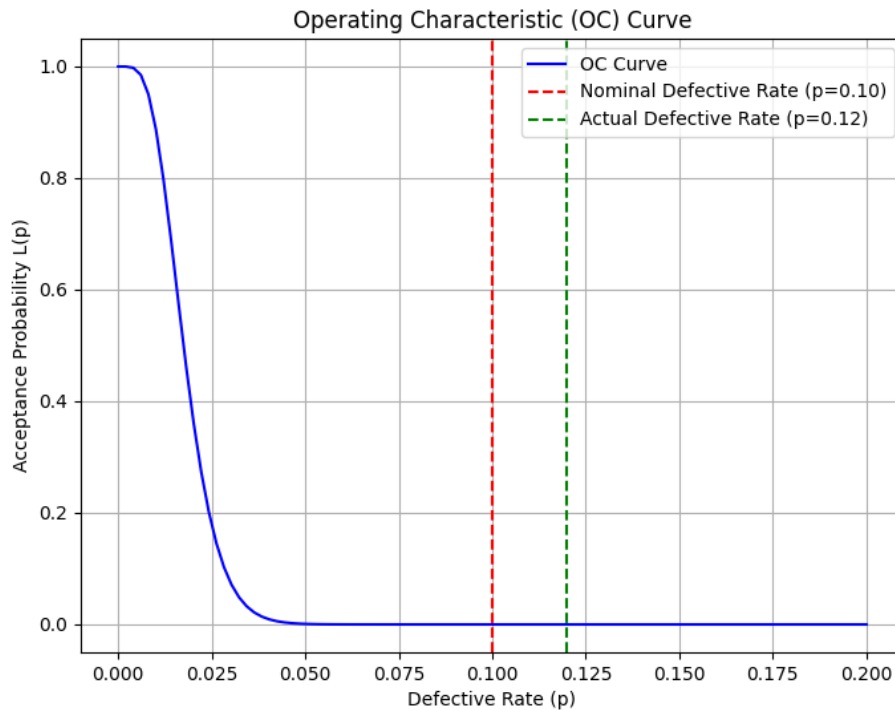


图 4 oc 曲线可视化

上图是 OC 曲线的图像，其中，横轴表示次品率 p ，范围从 0 到 20%。纵轴表示接受批次的概率 $L(p)$ ，即在给定次品率下，该批次零配件被接受的可能性。红色虚线表示标称次品率为 10%。这是企业或供应商声称的标准。绿色虚线表示假设的实际次品率为 12%。

由图可知，随着次品率增加，批次接受概率快速下降，符合逻辑。

标称次品率为 10%，实际为 12%，在 12% 下批次接受概率接近 0。说明抽样方案可以有效识别高次品率，12% 时几乎总拒收批次，检测效果良好。曲线在次品率 5% 附近拐点明显，能有效识别不合格产品。在低次品率下接受率高，方案宽容，但在 12% 情况下接受率几乎为 0。

总体来说，方案能有效拒绝高次品率批次，对 10% 次品率的灵敏度适中，对于 12% 及以上的次品率是有效的拒绝方案。

4.2 问题二模型的建立与求解

第二问是在已知的次品率基础上，做出最优的各阶段决策，是一个优化问题。

产品成品的大致流程如下：



图 5 产品成品流程图

基于前面的分析，模型的目标是通过合理的决策来求解工厂得最大利润。我们需要综合考虑各类成本，包括零配件采购成本、检测成本、成品装配成本、调换损失和拆解费用。我们将总成本记为：

$$\text{总成本} = \text{零件采购成本} + \text{零件装配成本} + \text{成品损失费} + \text{拆解费用}$$

其中，无论零配件是否检测或拆解，零件采购成本都是固定的；成品损失包括成品检测成本和调换损失，由于采购是整批购买，无论零件是否损坏，都需付出采购成本，所以不需要计算零件损失费。

主要决策变量有：

- (1) x_1 : 是否检测零件 1, $x_1 = 1$ 表示检测, $x_1 = 0$ 表示不检测。
- (2) x_2 : 是否检测零件 2, $x_2 = 1$ 表示检测, $x_2 = 0$ 表示不检测。
- (3) x_3 : 是否检测成品, $x_3 = 1$ 表示检测, $x_3 = 0$ 表示不检测。
- (4) x_4 : 是否拆解不合格成品, $x_4 = 1$ 表示拆解, $x_4 = 0$ 表示不拆解。

下面进行各部分成本进行分析与求解：

1. 零配件采购和检测成本

无论零配件是否检测，采购成本都是固定的，分别为 C_1 和 C_2 。如果选择对零配件进行检测，还会产生检测费用。

零配件 1 成本：

$$C'_1 = C_1 \cdot S_1 + x_1 \cdot C_{t1} \cdot S_1 \quad (4)$$

零配件 2 成本：

$$C'_2 = C_2 \cdot S_1 + x_2 \cdot C_{t2} \cdot S_1 \quad (5)$$

其中,

C_1 是零配件1的购买单价, C_2 是零配件2 的购买单价, S_1 是进货数量。

2. 装配成本

装配成本则需要考虑进入装配环节的零件数量,因为检测后会将次品零件丢弃,所以检测与非检测的零件批次进入装配环节的零件数量不相同。装配数量 S_2 求解公式如下:

$$S_2 = \min(S_1 \times (1 - x_1 \cdot d_1), S_1 \times (1 - x_2 \cdot d_2)) \quad (6)$$

其中, S_2 是装配数量。

那么, 装配成本按装配数量计算, 为:

$$C_{\text{零装}} = C_{\text{装}} \times S_2 \quad (7)$$

3. 成品检测和次品调换损失

成品次品率 d_t 由零配件次品率和成品合成次品率共同决定, 公式如下:

$$d_t = (1 - x_1)(1 - x_2)[1 - (1 - d_1)(1 - d_2)(1 - d_3)] + (1 - x_1)x_2[1 - (1 - d_1)(1 - d_3)] + x_1(1 - x_2)[1 - (1 - d_2)(1 - d_3)] + x_1x_2(1 - d_3) \quad (8)$$

成品损失包含成品检测成本和调换损失:

正品成品数量 $S_{\text{正}}$ 等于装配后的数量扣除次品成品数量:

$$S_{\text{正}} = S_2 \cdot (1 - d_t)$$

成品的检测成本为:

$$C_{t3} = X_3 \cdot C_{t3} \cdot C_{\text{总装}} \quad (9)$$

其中, C_{t3} 是成品检测总成本, C_{t3} 是一个成品检测的费用。

成品调换损失:

如果成品未经过检测, 次品进入市场后会产生调换损失, 计算公式为:

$$C_{\text{调}} = (1 - x_3) \times d_t \times C_{\text{调}} \times C_{\text{总装}} \quad (10)$$

4. 拆解成本和拆解收益

拆解收益: 通过拆解不合格成品, 可以回收正品零配件, 拆解收益计算为:

$$C_{\text{拆收}} = P(\text{正品零件 1, 次品零件 2}) \cdot C_1 + P(\text{次品零件 1, 正品零件 2}) \cdot C_2 + P(\text{正品零件 1, 正品零件 2}) \cdot (C_1 + C_2) \quad (11)$$

拆解成本: 拆解不合格成品会产生成本, 计算公式为:

$$C_{\text{拆总}} = x_4 \cdot C_{\text{拆}} \cdot C_{\text{装}} \quad (12)$$

5. 收入的计算

企业的总收入不仅包括正品成品的销售收入，还包括拆解后的收益：

$$C_{\text{总}} = S_{\text{正}} \times C_{\text{市}} + C_{\text{拆总}} \quad (13)$$

由上述分析，得到**目标函数** (即利润最大，利润等于总收入减去总成本) 为：

$$\max P_{\text{润}} = C_{\text{总}} - (C_1 + C_2 + C_{\text{总装}} + C_{t3} + C_{\text{调}} + C_{\text{拆总}}) \quad (14)$$

约束条件为： $x_1, x_2, x_3, x_4 \in \{0, 1\}$ (决策变量均为 0 或 1 的二元变量^[1])。

企业的目标是通过调整检测和拆解策略 x_1, x_2, x_3, x_4 ，使利润 P 最大化。

上述公式 (16) 中， C_1 是零配件1的购买单价； C_2 是零配件2 的购买单价； d_1 是零配件1 的次品率； d_2 是零配件2 的次品率； C_{t1} 是零配件1 的检测成本； C_{t2} 是零配件2 的检测成本； $C_{\text{装}}$ 是成品装配成本； d_3 是成品合成的次品率； d_t 是成品的次品率； C_{t3} 是成品检测成本； $C_{\text{市}}$ 是市场售价； $C_{\text{调}}$ 是调频损失； $C_{\text{拆}}$ 是拆解费用。

下面对表 1 企业在生产中遇到的 6 种情况做分析，根据表 1 中的数据进行计算：

为了找到最优的检测方案，我们需要对所有 x_1, x_2, x_3, x_4 的组合进行测试，并计算每种组合下的总利润，进而选择总利润最大的方案。

共有 2^4 种不同的 x_1, x_2, x_3, x_4 组合。接下来，本文将计算每种情况下的总利润最大值，并给出最优检测方案。

本文以表 1 第 1 种情况为例，计算出它的最大总利润：

基于产品生产的决策树，对每个路径及其对应的总利润进行详细分析。

在该回归树中， x_1, x_2, x_3, x_4 分别表示决策变量：

x_1 ：是否检测零配件 1， $x_1 = 1$ 表示检测， $x_1 = 0$ 表示不检测。

x_2 ：是否检测零配件 2， $x_2 = 1$ 表示检测， $x_2 = 0$ 表示不检测。

x_3 ：是否检测成品， $x_3 = 1$ 表示检测， $x_3 = 0$ 表示不检测。

x_4 ：是否拆解成品， $x_4 = 1$ 表示拆解， $x_4 = 0$ 表示不拆解。

我们求出部分策略组合：

策略 (x1, x2, x3, x4)	利润 (元)	总收入 (元)	总成本 (元)	正品成品数量 (件)	装配数量 (件)
(1, 1, 0, 0)	12,423.76	45,360	32,936.24	810	900
(0, 0, 0, 0)	11,201.76	40,824	29,622.24	729	1,000
(1, 0, 0, 0)	10,401.76	40,824	30,422.24	729	900

图 6 部分的策略组合

总结：

• 最优策略是检测零配件 1 和零配件 2，但不对成品进行检测和拆解，这可以获得最大利润 12,423.76 元。

• 较差策略是对成品和零配件进行大量检测并进行拆解, 导致成本较高, 利润较低。

通过对所有 x_1, x_2, x_3, x_4 组合的计算, 得到了情况 1-6 的最大利润及其对应的最优决策方案:

情况	策略 (x_1, x_2, x_3, x_4)	利润 (元)	总收入 (元)	总成本 (元)	正品成品数量 (件)	装配数量 (件)
1	(1, 1, 0, 0)	12,423.76	45,360	32,936.24	810	900
2	(1, 1, 0, 0)	3,086.34	35,840	32,753.66	640	800
3	(1, 1, 0, 0)	10,263.76	45,360	35,096.24	810	900
4	(1, 1, 1, 0)	5,446.34	35,840	30,393.66	640	800
5	(0, 1, 0, 0)	6,971.74	36,288	29,316.26	648	800
6	(0, 0, 0, 0)	18,588.79	48,013	29,424.21	857	1,000

图 7 问题二各情况最优策略

4.3 问题三模型的建立与求解

问题 3 扩展了问题 2 的情境, 增加了复杂度, 涉及多个零配件和多道工序的生产过程。如图所示:

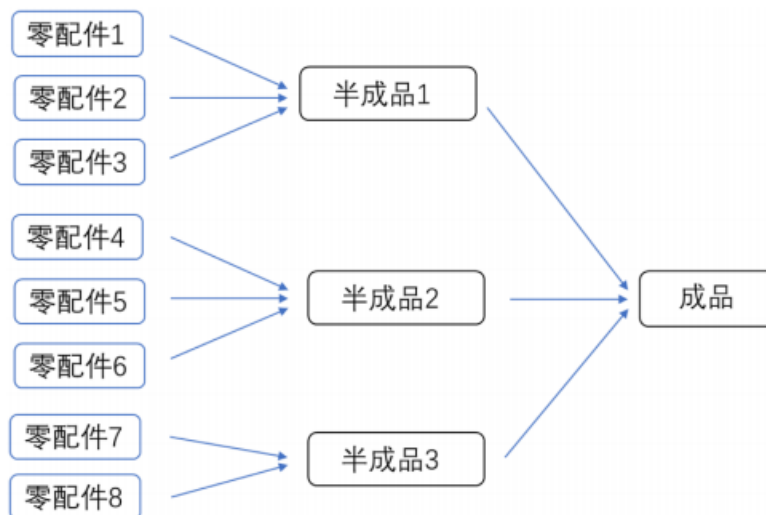


图 8 问题三组装情况

问题要求在已知零配件、半成品和成品的次品率的情况下, 为整个生产过程设计优化决策方案。

工厂需要对每一个生产阶段做出优化决策, 目的是在利润最高的情况下, 保证尽可能多的合格产品进入市场。

一. 模型符号的定义

为了建立数学模型, 本文首先定义了一些符号:

零件符号

C_i : 第 i 个零件的采购成本。

C_{it} : 第 i 个零件的检测成本。

d_i : 第 i 个零件的次品率。

X_i : 第 i 个零件的检测决策, $X_i = 1$ 表示检测, $X_i = 0$ 表示不检测。

半成品符号

d_{t1}, d_{t2}, d_{t3} : 分别为半成品 1、2、3 的总次品率, 包括零件次品率和正品零件合成次品率。

d_{b1}, d_{b2}, d_{b3} : 分别为正品零件 1、2、3, 零件 4、5、6, 零件 7、8 合成半成品时的次品率。

X_{s1}, X_{s2}, X_{s3} : 分别表示半成品 1、2、3 是否进行检测, $X_{sj} = 1$ 表示检测, $X_{sj} = 0$ 表示不检测。

成品符号

d_f : 成品的次品率, 由三个半成品的次品率组合而成。

d_b : 正品半成品 1、2、3 合成成品时的次品率。

二. 目标函数的确立

1. 零件和半成品次品率的计算

(1) 零件检测与次品率的关系:

- 零件的检测会剔除次品, 因此检测后的零件次品率为 0, 正品率为 1。
- 未检测的零件次品率维持原有的值。假设 d_i 为零件 i 的次品率, x_i 为是否对零件 i 进行检测的决策变量 (1 表示检测, 0 表示不检测), 则经过检测后, 零件的次品率为:

$$d_i' = (1 - x_i) \cdot d_i$$

正品率为:

$$p_i' = x_i + (1 - x_i)(1 - d_i)$$

其中 p_i' 表示经过检测后的有效正品率。

(2) 半成品次品率的计算:

半成品由多种零件组成, 假设零件 1、2、3 组成半成品 1, 零件 4、5、6 组成半成品 2, 零件 7、8 组成半成品 3。则半成品次品率依赖于其组成零件的检测和正品率。

- 半成品 1 的次品率:

$$d_{t1} = 1 - p_1' \cdot p_2' \cdot p_3'$$

- 半成品 2 的次品率:

$$d_{t2} = 1 - p_4' \cdot p_5' \cdot p_6'$$

- 半成品 3 的次品率:

$$d_{t3} = 1 - p_7' \cdot p_8'$$

(3) 成品次品率的计算:

成品由半成品 1、2、3 组合而成, 其次品率为:

$$d_f = 1 - (1 - d_{t1}) \cdot (1 - d_{t2}) \cdot (1 - d_{t3})$$

2. 数量计算

(1) 零件有效数量:

对零件进行检测后, 剔除次品, 因此检测后的有效数量为:

$$N_{\text{效}i} = N_i \cdot p_i'$$

(2) 半成品数量:

半成品数量由其组成零件的有效数量中最小的那个决定。例如，半成品 1 由零件 1、2、3 组成，半成品数量 N_i 为：

$$N_1 = \min(N_{\text{效}1}, N_{\text{效}2}, N_{\text{效}3})$$

同理计算半成品 2 和半成品 3 的数量。

(3) 成品数量：

成品数量由半成品 1、2、3 的有效数量中最小的那个决定：

$$N_f = \min(N_1, N_2, N_3)$$

3. 检测成本和拆解收益

(1) 检测成本：

半成品和成品的检测成本应分别计算。例如，假设对半成品 1 进行检测，其检测成本为：

$$C_{bt1} = C_{bt1} \cdot N_i$$

其中， C_{bt1} 是半成品 1 的检测总费用， C_{bt1} 是 1 个半成品 1 的检测费用， N_i 是半成品 1 的检测数量。

成品检测成本为：

$$C_{\text{总}t} = C_{tf} \cdot N_f$$

其中， $C_{\text{总}t}$ 是成品检测的总成本， C_{tf} 是一个成品检测的费用， N_f 是检测成品的数量。

(2) 拆解收益：

拆解后的正品零件可以再次使用或出售，拆解收益应按拆解后的正品零件的数量和单价计算

解后的正品数量为：

$$N_{\text{拆正}} = \sum_{i=1}^8 P(\text{正品零件}_i) \cdot N_i$$

拆解收益为：

$$C_{\text{拆收}} = \sum_{i=1}^8 C_i \cdot N_{\text{拆正}i}$$

其中， $C_{\text{拆收}}$ 是拆解所得收益， $N_{\text{拆正}i}$ 是拆解的零件 i 的数量。 C_i 是零件 i 的进货价。

通过引入半成品的检测成本、修正零件次品率的计算方式以及拆解收益的计算，模型能够更精确地反映整个生产过程中的各项成本和收益。目标是通过优化检测和拆解策略，最大化利润。

最终，我们的目标函数更改为：

$$\max P_{\text{润}} = (N_f \cdot C_{\text{市}} + C_{\text{拆收}}) - (C_{\text{采}} + C_{\text{bt}} + C_{\text{半拆}} + C_{\text{调}} + C_{\text{拆总}} + C_{\text{总检测}}) \quad (15)$$

其中， $C_{\text{采}}$ 是零件的采购和检测成本之和， C_{bt} 是半成品的检测总费用， $C_{\text{半拆}}$ 半成品的拆解总费用 $C_{\text{调}}$ 成品的调换总成本， $C_{\text{拆总}}$ 成品的拆解总成本， $C_{\text{总检测}}$ 成品的

检测总成本。它们具体的求解方程，分别与公式 (6)、(7); (11); (14);(12);(14);(11) 类似。

三. 模型求解

本文使用 matlab 软件，对各种情况进行遍历，共有 2^{13} 种组合，算出利润最大的决策方案为：零件 1-8 均进行检验，半成品均不进行检验，半成品不进行拆解，成品不进行检测，对次品成品进行拆解。

四. 模型优化

可以尝试使用智能优化算法如遗传算法或者蒙特卡洛算法等简化计算过程，而不是对全部情况都进行遍历。

4.4 问题四模型的建立与求解

假设问题 2 和问题 3 中零配件、半成品和成品的次品率均是通过序贯概率比检测方法抽样检测得到的，下面在第 1、2、3 问的基础上对模型进行改进重新完成问题 2 和问题 3。

通过 SPRT 抽样检测更新次品率，本文设计了一套动态调整生产策略的方法，以减少次品率对企业成本和收益的影响。改进后的模型综合考虑了检测、装配、拆解等环节的动态调整策略，有效优化了企业的生产流程和成本控制。

一. 零配件和成品的抽样检测 (SPRT 应用)

我们首先利用 SPRT 对零配件 1、零配件 2 和成品进行抽样检测，动态估计其次品率。每次检测后，我们计算得到更新后的次品率 p_{new} ，并将其反馈至生产流程的各个环节。

假设：

(1) 对零配件 1 和 2 进行 SPRT 抽样检测

(2) 对成品讲行 SPRT 抽样检测

通过每次抽样，更新火品率 p_{new} ，用于生产过程中的决策调整。

二. 成品次品率的动态计算：

次品率的估计值更新后，成品次品率 d_t 也需要动态调整。以下是次品率的不同计算情况：

如果检测了零配件 1 和 2：

$$d_t = d_{\text{测}}$$

如果只检测了零配件 2：

$$d_t = (1 - d_1) \cdot d_{\text{测}} + d_1$$

如果未检测零配件 1 和 2：

$$d_t = 1 - (1 - d_1) \cdot (1 - d_2) \cdot (1 - d_{\text{测}})$$

三. 检测、装配与拆解策略的动态调整

根据 SPRT 更新的次品率，我们将对以下几个环节进行动态调整

1. 检测策略调整与拆解策略调整:

1.1 零件检测调整:

- 当 SPRT 估计出较高的次品率时, 应加大检测力度。例如, 针对零配件的次品率, 如果 SPRT 估计的次品率较高 (如超过 10%), 应对零配件和成品进行更多的检测。

- 当 SPRT 估计的次品率较低时, 可以减少检测, 降低检测成本。

1.2 成品检测调整:

如果 SPRT 估计成品的次品率较高, 则对装配后的成品进行检测, 以避免次品流入市场。如果次品率较低, 企业可以选择不检测成品, 从而节省检测成本。

(1) 零配件数量较少 (例如 100 件以内)

对于小规模生产。检测成本在总成本中占比相对较高, 企业可能会选择减少检测, 除非次品率估计较高。

- 决策: 如果次品率较低 (例如 $< 10\%$), 则不对零配件 1 和零配件 2 进行检测, 直接进入装配, 如果次品率较高, 则对成本较高的零配件进行检测 (例如检测零配件 2)。

- 不检测条件: $p_{\text{new}} \leq 10\%$, 不检测零配件。

- 检测条件: $p_{\text{new}} > 10\%$, 检测成本较高的零配件。

装配后决策: 如果零配件经过检测目成品次品率低, 则不检测成品直接销售; 如果成品次品率较高, 则对成品进行抽样检测, 减少次品流入市场。

(2) 中等规模生产 (100 到 1000)

在这种情况下, 次品率的波动对总成本的影响较大, 企业应根据检测和拆解的回收收益做出更复杂的决策。

- 决策: 如果次品率低于 10%, 可以减少检测零配件的频率, 只检测其中一个 (检测零配件 2, 因其成本更高); 如果次品率超过 10%, 建议对两个零配件进行检测, 并对成品进行随机抽样检测。

- 检测条件: $p_{\text{new}} > 10\%$, 检测零配件 1 和零配件 2。

- 不检测条件: $p_{\text{new}} \leq 10\%$, 仅检测零配件 2。

拆解决策: 如果成品的次品率 d_{total} 大于某个阈值 (例如 $d_{\text{total}} \geq 20\%$), 且拆解后的回收收益高于拆解成本, 则进行拆解; 否则不拆解。

- 拆解条件: 如果拆解收益 $>$ 拆解成本, 则进行拆解。

- 不拆解条件: 如果拆解收益 $\leq$ 拆解成本, 则不拆解, 直接丢弃不合格成品。

(3) 大规模生产 (1000 件及以上)

在大规模生产中, 次品率对总收益和利润的影响尤为显著, 企业需要更谨慎地控制次品流入市场。对于大规模生产, 检测和拆解成本在总成本中占比变小, 因此企业可能需要进行更多的检测和拆解操作, 以减少次品带来的损失。

- 决策: 当生产规模较大时, 如果次品率估计较高 (例如超过 15%), 企业应全面检测零配件 1 和零配件 2, 同时也应对成品进行全面检测。这样做虽然增加了检测成本, 但可以有效减少次品的市场流通, 避免高额的调换损失。

2. 装配数量的调整:

根据经过检测后的零配件 1 和零配件 2 的数量, 动态调整装配后的数量。装配数量的计算应为:

$$\text{装配数量} = \min(\text{零配件 1 数量}, \text{零配件 2 数量})$$

装配数量是受检测后合格零配件数量约束的, 如果两种零配件中的某一种次品率过高, 可能导致装配数量的减少。

装配数量会随着 SPRT 检测结果中的次品率变化而变化。

根据 SPRT 估计的成品次品率 d_t , 评估拆解的收益与成本。如果成品次品率较高, 并且拆解后能够回收有价值的零配件, 建议进行拆解; 否则, 选择不拆解。

3. 动态成本与收益的计算

通过进行动态次品率估计后, 成本与收益地将根据生产过程中的各个决策实时调整。模型中涉及的动态调整策略不仅优化了生产环节, 还降低了因次品流入市场所带来的损失。

- 检测条件: $p_{\text{new}} > 15\%$, 全面检测零配件 1、零配件 2 和成品。
- 不检测条件: $p_{\text{new}} \leq 15\%$, 抽样检测零配件或成品。
- 装配后决策: 如果装配后的成品次品率较高 (如 $d_t \geq 20\%$), 企业应对成品进行拆解, 以回收可用的零配件。如果次品率较低, 则不拆解直接进入市场。
- 拆解条件: 拆解收益大于成本, 则拆解。
- 不拆解条件: 拆解收益不高于成本, 则丢弃不合格品。

五、模型评价

一. 模型的优点:

1. **节省样本量:** SPRT 通常比固定样本量的检验方法更有效率, 能够在较少的数据点下得出显著结论, 节省实验或研究的成本, 适配题目检验次数尽量少的要求。
2. **适应性强:** 序贯概率比检验可以根据研究者的需求设定不同的显著性水平和功效, 灵活调整检验策略。
3. **动态决策:** SPRT 模型可以在检测过程中动态做出接收或拒收决策, 而无需一次性完成所有样本检测。这显著减少了所需样本量, 提升了效率。

二. 模型的缺点:

依赖于正确的模型假设: SPRT 假设了两个精确的假设 (如 H_0 和 H_1), 而这通常要求我们对数据分布有很好的了解。如果假设模型不正确或数据分布未知, SPRT 的性能会受到显著影响。

六、模型检验

一. 问题一模型检验对于问题一模型检验, 本文引入评卷检验量、平均检出质量对模型进行评估。

1. **平均检验量 (Average Number of Inspections, ATI):**

平均检验量是一个经济指标，用于衡量抽样检验的成本效益。它基于假设：对于被拒收的批次，生产方会进行全检并替换不合格品。ATI 的计算考虑了抽样数量 n 和总产品数量 N ，以及批次被接收的概率 $L(p)$ 。

其计算公式为：

$$ATI = nL(p) + N[1 - L(p)] = n + (N - n)[1 - L(p)] \quad (16)$$

其中， n 是抽样方案中抽取的样本数， $L(p)$ 是当批产品的合格率为 p 时，产品被接收的概率， N 是每批产品的总数量。

2. 平均检出质量 (Average Outgoing Quality, AOQ) :

平均检出质量反映了接收批次的平均质量水平，即不合格品在全部被检产品中所占的比例。其计算公式为：

$$AOQ = pL(p) \quad (17)$$

这里， p 是批产品的次品率， $L(p)$ 是产品被接收的概率。该公式考虑了检验过程中接收的批次数量及其中的不合格品数量，为评估接收批次的质量水平提供了量化指标。

将平均检验量与平均检出质量做可视化处理，可以帮助我们更直观地看出抽样模型的特性。由下图：

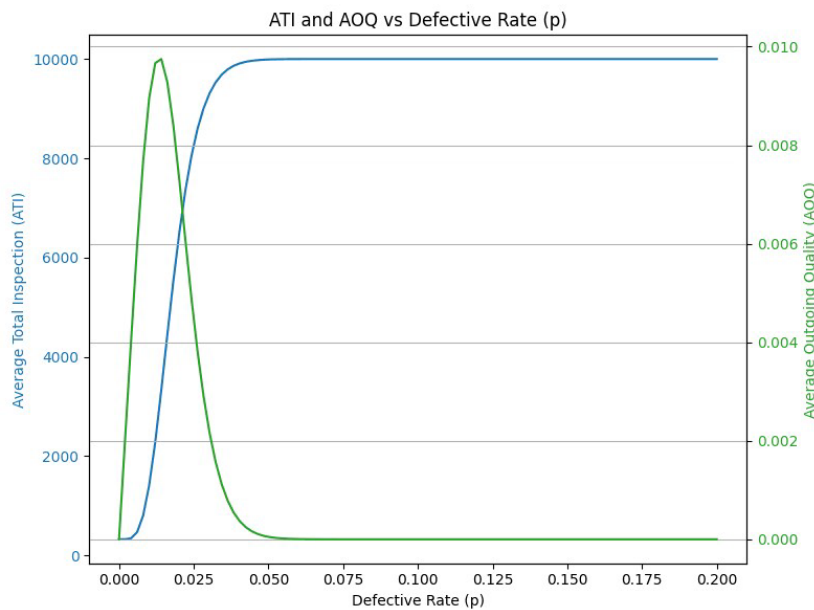


图 9 平均检验量和平均检出质量

当次品率较低时，AOQ 相对较高，ATI 相对较低，说明在这种情况下系统输出的产品质量不高，需要进行更多的检验。

随着次品率增加，ATI 增加，意味着需要更多的检验。图中的平均总检验次数 (ATI) 接近于一万次，可能是由于系统对质量要求严格，即便是次品率很低的情况下，依然会进行大量的检验。

当次品率非常高时，AOQ 接近 0，ATI 趋于稳定，表明系统在高缺陷率时已经能够通过检验进行有效控制。

通过以上推导和分析, 本文建立了一个基于次品率的序贯检测模型, 该模型不仅具有良好的统计功效, 还可以灵活地适应不同生产批次中的质量控制需求。通过构建和计算似然比, 并引入合适的偏差值, 可以有效地决定是否接受或拒绝某批次的零配件。同时, 基于数值模拟的分析可以进一步验证该模型在实际应用中的有效性。

二. 问题三模型检验

对于这两张图可以进行以下分析:

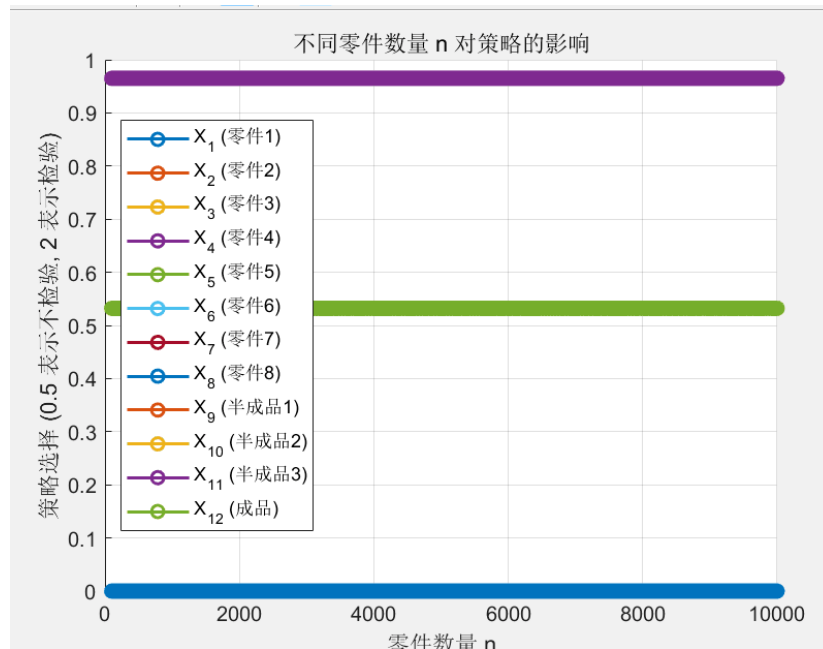


图 10 不同零件数量 n 对策略的影响

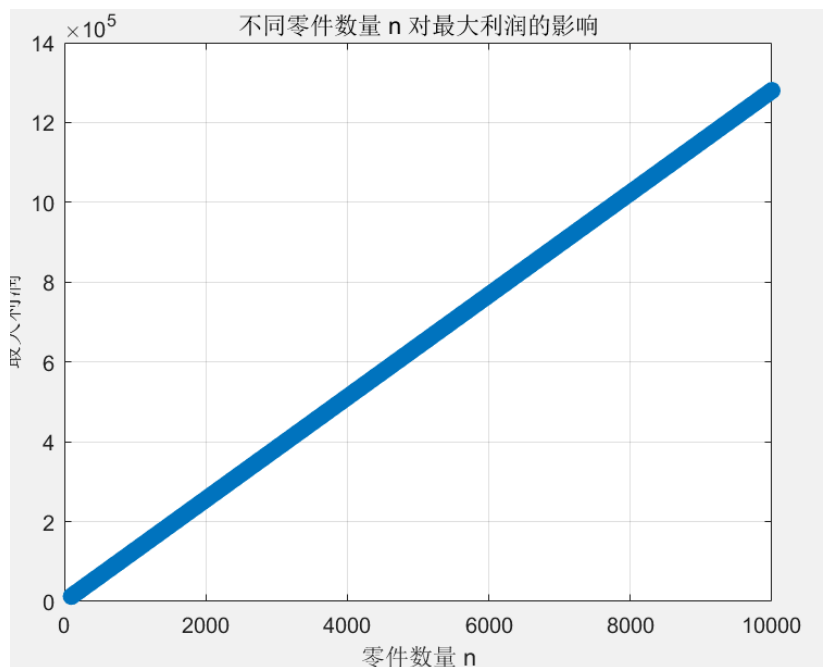


图 11 不同零件数量 n 对最大利润的影响

第一张图 (变化剧烈的策略图):

- 这张图展示了零件数量 n 在从 1000 到 9000 之间变化时, 各个决策变量 (检测

与不检测) X_1 到 X_{12} 的波动情况。

- 其中, 绿色的曲线 X_{12} 表示成品的检验与否, 其变化非常频繁, 说明随着零件数量的变化, 成品是否检验的决策不稳定, 这可能是由于成品检验和利润之间的非线性关系。

- 其他的一些零件和半成品检验策略也表现出波动, 尤其是 X_3 、 X_6 等零件的检验策略波动较大。这说明在不同零件数量下, 某些零件或半成品的检验策略频繁调整, 以寻找最优的利润方案。这种波动性可能与利润最大化的需求相关, 随着零件数量的变化, 某些零件或半成品的检验决策不再固定, 而是基于当前的数量情况动态调整。

第二张图 (稳定的策略图):

- 这张图展示了一个更加稳定的策略变化情况。随着零件数量 n 从 1000 到 9000 变化, 大部分变量几乎保持稳定, 不同的检测策略选择几乎不变。

- 在该图中, 决策变量 X_6 保持在 0.5, 这意味着零件 6 在这个范围内始终不被检验, 而其他变量, 诸如 X_4 (紫色) 和 X_8 (蓝色) 也保持在固定的状态, 分别是 2 和 0.5。

- 这种情况下, 最优策略似乎在这些数量范围内几乎不变, 说明模型在这些区间内的策略已经找到了一种稳定的最优解, 不再需要根据数量变化调整策略。

总结:

第一张图展现了较大范围内不同零件数量对策略的动态影响, 这些波动说明在不同的零件数量下, 策略需要不断调整以找到利润最大化的方案。

第二张图则说明在某些情况下, 策略趋于稳定, 随着零件数量的增加, 最优的检测决策不再频繁变化, 模型找到了稳定的策略来处理这些情况。两张图的区别也表明, 随着零件数量增加, 有时候策略会趋于稳定, 而在某些区间, 策略会根据具体情况进行动态调整。

从图中可以看出, 最大利润随着零件数量的增加呈现线性增长趋势。这说明在不同的零件数量下, 最大利润基本上是单调递增的。这种线性关系表明, 当前情况下零件数量越多, 系统生产的利润越高。但是, 这并不能完全证明最大利润率只是一种最优策略, 具体来说:

1. 线性增长趋势: 图中的线性增长趋势表明, 在模型假设的参数和条件下, 随着零件数量 n 增加, 利润在逐步提升。而最优策略则是根据这些输入条件 (零件数量、次品率等) 推导出的结果。

2. 最优策略: 这里的“最优策略”可能并不是唯一的, 也就是说, 在不同的零件数量下, 虽然最大利润是递增的, 但不同策略的选择可能会有所不同。例如, 当零件数量较少时, 可能需要检验更多的零件和半成品, 而在数量足够多时, 检验的必要性可能下降。

3. 利润增速: 线性增长也表明当前的生产规模没有遇到瓶颈或制约条件。换句话说, 生产成本没有随着零件数量的增加而导致效率降低。因此, 模型中的最优策略适用于当前规模。

4. 策略的变化: 尽管图中最大利润随零件数量增加, 但要深入分析每个零件、半成品或成品的检测策略是否随着数量变化而发生了改变。最优策略在某些数量范围内可能是相同的, 但随着数量增加, 某些零件的检验可能会减少或增加, 以适应最大利

润。

结论：这张图证明了当前情况下的最大利润呈现线性增长，但是否唯一的最优策略，还需要进一步通过策略的变化分析。例如，绘制零件数量与各检测策略（如零件、半成品、成品是否检验）的关系图，来分析随着数量变化，检测策略的选择是否发生变化。你可以通过策略曲线分析是否有多个最优策略，并进一步优化生产管理。

七、参考文献

[1] 马志远. 基于序贯概率比检验和小波神经网络的齿轮箱故障诊断方法 [D]. 武汉大学,2016.

[2] 于春田. 零一规划在投资优化中的应用研究 [J]. 河北轻化工学院学报,1994,(03):65-69.

[3] 杨风萍. 带有检测风险的抽样方案设计及其应用 [D]. 南京信息工程大学,2023.DOI:10.27248/d.cnki.gnjqc.2023.001003.

八、附录

表 2 附录目录

内容
问题一代码
问题二代码与结果
第三问代码
问题三模型检验 matlab 代码

A附录一：问题一代码

```
1 偏差值与总体量的讨论python代码
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # SPRT 检验参数
6 def sprt_test(p0, p1, alpha, beta, max_samples, batch_size=1):
7     """
8     SPRT序贯概率比检验的实现。
9
10    参数:
11    - p0: 原假设下的次品率
```



```

12 - p1: 备择假设下的次品率
13 - alpha: 一类错误率 (显著性水平)
14 - beta: 二类错误率 (功效水平)
15 - max_samples: 最大检测样本数
16 - batch_size: 每次检测的样本大小
17
18 返回:
19 - result: "Accept H0" 或 "Reject H0"
20 - samples: 进行的总样本数
21 - likelihood_ratios: 每次检测后的累积似然比
22 """
23
24 # 计算决策阈值
25 A = (1 - beta) / alpha # 拒绝H0
26 B = beta / (1 - alpha) # 接受H0
27
28 # 初始化变量
29 likelihood_ratio = 1
30 total_samples = 0
31 likelihood_ratios = []
32
33 # 模拟检测过程
34 while total_samples < max_samples:
35     # 随机生成一批样本, 假设真实次品率为 p1
36     defects = np.random.binomial(batch_size, p1)
37
38     # 计算当前的似然比
39     likelihood_ratio *= (p1 ** defects * (1 - p1) ** (batch_size - defects)) / \
40     (p0 ** defects * (1 - p0) ** (batch_size - defects))
41
42     # 更新样本数
43     total_samples += batch_size
44     likelihood_ratios.append(likelihood_ratio)
45
46     # 根据似然比判断是否停止
47     if likelihood_ratio >= A:
48         return "Reject H0", total_samples, likelihood_ratios
49     elif likelihood_ratio <= B:
50         return "Accept H0", total_samples, likelihood_ratios
51
52     return "Continue Sampling", total_samples, likelihood_ratios
53
54 # 固定的原假设次品率
55 p0 = 0.10 # 原假设次品率 (标称次品率)
56 alpha = 0.05 # 显著性水平 (第一类错误率)
57 beta = 0.1 # 第二类错误率 (功效水平)
58 batch_size = 1 # 每次检测的样本大小
59
60 # 偏差参数列表
61 deltas = np.arange(0.005, 0.051, 0.005) # 从 0.005 到 0.05, 步长为 0.005
62
63 # 不同的样本量上限
64 sample_limits = [1000, 2000, 5000, 10000] # 分别为1000、2000、5000和10000

```

```

65
66 # 初始化结果存储
67 results = np.zeros((len(deltas), len(sample_limits)))
68
69 # 对每个偏差值和不同样本量运行SPRT并记录所需样本数
70 for i, delta in enumerate(deltas):
71     p1 = p0 + delta # 调整备择假设次品率
72     for j, max_samples in enumerate(sample_limits):
73         result, samples_used, __ = sprt_test(p0, p1, alpha, beta, max_samples, batch_size)
74         results[i, j] = samples_used # 记录所需样本数
75     print(f"偏差: {delta:.3f}, 样本量上限: {max_samples}, 使用的样本总数: {samples_used}, 结果: {result}")
76
77 # 绘制结果图
78 fig, ax = plt.subplots()
79
80 # 针对不同的样本量上限，绘制每个偏差下的样本数曲线
81 for j, max_samples in enumerate(sample_limits):
82     ax.plot(deltas, results[:, j], label=f'样本量上限 {max_samples}')
83
84 # 设置图表的标签和标题
85 ax.set_xlabel('偏差 ( )')
86 ax.set_ylabel('所需样本数')
87 ax.set_title('不同样本量上限和偏差下的所需样本数')
88 ax.legend()
89 plt.grid(True)
90 plt.show()

```

```

1  偏差量0.03，总体量10000时的样本量计算python代码
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def sprt_test(p0, p1, alpha, beta, max_samples, batch_size=1):
5
6      A = (1 - beta) / alpha
7      B = beta / (1 - alpha)
8
9      likelihood_ratio = 1
10     total_samples = 0
11     likelihood_ratios = []
12     while total_samples < max_samples:
13         defects = np.random.binomial(batch_size, p1)
14         likelihood_ratio *= (p1 ** defects * (1 - p1) ** (batch_size - defects)) / \
15             (p0 ** defects * (1 - p0) ** (batch_size - defects))
16
17     total_samples += batch_size
18     likelihood_ratios.append(likelihood_ratio)
19
20     if likelihood_ratio >= A:
21         return "Reject H0", total_samples, likelihood_ratios
22     elif likelihood_ratio <= B:
23         return "Accept H0", total_samples, likelihood_ratios
24
25     return "Continue Sampling", total_samples, likelihood_ratios

```

```

26 p0 = 0.10
27 alpha = 0.05
28 beta = 0.1
29 batch_size = 1
30
31
32 deltas = np.arange(0.005, 0.051, 0.005)
33 sample_limits = [1000, 2000, 5000, 10000]
34 results = np.zeros((len(deltas), len(sample_limits)))
35 for i, delta in enumerate(deltas):
36     p1 = p0 + delta
37     for j, max_samples in enumerate(sample_limits):
38         result, samples_used, _ = sprt_test(p0, p1, alpha, beta, max_samples, batch_size)
39         results[i, j] = samples_used
40     print(f"偏差: {delta:.3f}, 样本量上限: {max_samples}, 使用的样本总数: {samples_used}, 结果: {result}")
41
42 fig, ax = plt.subplots()
43 for j, max_samples in enumerate(sample_limits):
44     ax.plot(deltas, results[:, j], label=f"样本量上限 {max_samples}")
45     ax.set_xlabel('偏差 ( )')
46     ax.set_ylabel('所需样本数')
47     ax.set_title('不同样本量上限和偏差下的所需样本数')
48     ax.legend()
49 plt.grid(True)
50 plt.show()

```

B附录二：问题二代码及结果

```

1 产品生产流程图代码
2 import matplotlib.pyplot as plt import networkx as nx
3
4
5 plt.rcParams['font.sans-serif']=['SimHei'] #使用黑体
6 plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
7
8 G_updated_vars_2 = nx.DiGraph()
9
10
11 G_updated_vars_2.add_node("购买零件1 (C_1)").add_node("购买零件2 (C_2)")
12 G_updated_vars_2.add_node("检验零件1-正品
    (C_{t1})")G_updated_vars_2.add_node("检验零件1-次品
    (C_{t1})")G_updated_vars_2.add_node("不检验零件1")G_updated_vars_2.add_node("检验零件2-正品
    (C_{t2})")G_updated_vars_2.add_node("检验零件2-次品
    (C_{t2})")G_updated_vars_2.add_node("不检验零件2")G_updated_vars_2.add_node("丢弃零件1次品")G_updated_v
13
14
15 G_updated_vars_2.add_node("装配成品 (C_{装})")
16
17
18 G_updated_vars_2.add_node("成品检验-正品 (C_{t3})")G_updated_vars_2.add_node("成品检验-次品

```

```

(C_{t3})")G_updated_vars_2.add_node("不检验成品")G_updated_vars_2.add_node("市场检验-合格")G_updated_var
(C_{市})")G_updated_vars_2.add_node("市场退回 (C_{\text{调}})")

19
20
21 G_updated_vars_2.add_node("丢弃成品 (C_{\text{调}})")G_updated_vars_2.add_node("拆解
(C_{\text{拆}})")G_updated_vars_2.add_node("拆解后零件-重新进入流程")

22
23
24 G_updated_vars_2.add_edges_from([("购买零件1 (C_1)", "检验零件1-正品 (C_{t1})",
("购买零件1 (C_1)", "检验零件1-次品 (C_{t1})",
("购买零件1 (C_1)", "不检验零件1"), ("检验零件1-次品 (C_{t1})",
"丢弃零件1次品"))G_updated_vars_2.add_edges_from([("购买零件2 (C_2)", "检验零件2-正品
(C_{t2})", ("购买零件2 (C_2)", "检验零件2-次品 (C_{t2})",
("购买零件2 (C_2)", "不检验零件2"),
("检验零件2-次品 (C_{t2})", "丢弃零件2次品"))

25
26
27 G_updated_vars_2.add_edges_from([("检验零件1-正品 (C_{t1})", "装配成品 (C_{装})",
("不检验零件1", "装配成品 (C_{装}")),
("检验零件2-正品 (C_{t2})", "装配成品 (C_{装}")), ("不检验零件2",
"装配成品 (C_{装}"))])

28
29
30 G_updated_vars_2.add_edges_from([("装配成品 (C_{装})", "成品检验-正品 (C_{t3})",
("装配成品 (C_{装})", "成品检验-次品 (C_{t3}")),
("装配成品 (C_{装})", "不检验成品"))

31
32
33 G_updated_vars_2.add_edges_from([("成品检验-次品 (C_{t3})", "拆解 (C_{\text{拆}})",
("成品检验-次品 (C_{t3})", "丢弃成品 (C_{\text{调}})"))#
更新市场检验和市场退回流程G_updated_vars_2.add_edges_from([("成品检验-正品 (C_{t3})",
"市场检验-合格"), ("不检验成品", "市场检验-合格"),
("不检验成品", "市场检验-不合格"))

34
35
36 G_updated_vars_2.add_edges_from([("市场检验-不合格", "市场退回 (C_{\text{调}})",
("市场退回 (C_{\text{调}})", "拆解 (C_{\text{拆}})",
("市场退回 (C_{\text{调}})", "丢弃成品 (C_{\text{调}})"))

37
38
39 G_updated_vars_2.add_edges_from([("市场检验-合格", "流入市场 (C_{市}"))

40
41
42 G_updated_vars_2.add_edges_from([("拆解 (C_{\text{拆}})", "拆解后零件-重新进入流程",
("拆解后零件-重新进入流程", "检验零件1-正品 (C_{t1}")),
("拆解后零件-重新进入流程", "检验零件2-正品 (C_{t2}"))

43
44
45 pos_custom_final_vars_2 = { "购买零件1 (C_1)": (-1, 1), "检验零件1-正品 (C_{t1})": (-1.5, 0.5),
"检验零件1-次品 (C_{t1})": (-1.5, 0), "不检验零件1": (-0.5, 0.5), "丢弃零件1次品": (-2, 0),
"购买零件2 (C_2)": (1, 1), "检验零件2-正品 (C_{t2})": (1.5, 0.5), "检验零件2-次品 (C_{t2})": (1.5,
0), "不检验零件2": (0.5, 0.5), "丢弃零件2次品": (2, 0), "装配成品 (C_{装})": (0, 0.5),
"成品检验-正品 (C_{t3})": (-0.5, -1), "成品检验-次品 (C_{t3})": (0.5, -1), "不检验成品": (0, -0.5),

```

```

    "市场检验-合格": (-0.5, -1.5), "市场检验-不合格": (0.5, -1.5), "流入市场 (C_{市})": (-1, -2),
    "丢弃成品 (C_{\text{调}})": (1, -2), "市场退回 (C_{\text{调}})": (0, -2), "拆解 (C_{\text{拆}})":
    (0, -2.5), "拆解后零件-重新进入流程": (0, -3)}

46
47
48 plt.figure(figsize=(14, 10))nx.draw(G_updated_vars_2, pos=pos_custom_final_vars_2,
    with_labels=True, node_size=3000, node_color="lightblue", font_size=8, font_weight="bold",
    arrows=True)plt.title("流程图：替换变量符号 C市 并调小字号")plt.show()

49
50
51 # 在流程图中添加成本注释，并根据流程图进行标注
52
53 # 使用之前的流程图基础，添加相关成本注释
54
55 # 创建有向图
56
57 G_with_costs = nx.DiGraph()
58
59 # 添加零件相关节点G
60
61 _with_costs.add_node("购买零件1 (C_buyA)")G_with_costs.add_node("购买零件2
    (C_buyB)")G_with_costs.add_node("检验零件1-正品
    (C_checkA)")G_with_costs.add_node("检验零件1-次品
    (C_checkA)")G_with_costs.add_node("不检验零件1")G_with_costs.add_node("检验零件2-正品
    (C_checkB)")G_with_costs.add_node("检验零件2-次品
    (C_checkB)")G_with_costs.add_node("不检验零件2")G_with_costs.add_node("丢弃零件1次品")G_with_costs.add_node("丢弃零件2次品")

62
63
64 G_with_costs.add_node("装配成品 (C_assemble)")
65
66
67
68 G_with_costs.add_node("成品检验-正品 (C_checkC)")G_with_costs.add_node("成品检验-次品
    (C_checkC)")G_with_costs.add_node("不检验成品")G_with_costs.add_node("市场检验-合格")G_with_costs.add_node("市场检验-不合格")G_with_costs.add_node("市场退回 (L_change)")

69
70
71
72 G_with_costs.add_node("丢弃成品 (L_change)")G_with_costs.add_node("拆解
    (C_disassemble)")G_with_costs.add_node("拆解后零件-重新进入流程")

73
74
75
76 G_with_costs.add_edges_from([("购买零件1 (C_buyA)", "检验零件1-正品 (C_checkA)"),
    ("购买零件1 (C_buyA)", "检验零件1-次品 (C_checkA)"),
    ("购买零件1 (C_buyA)", "不检验零件1"), ("检验零件1-次品 (C_checkA)",
    "丢弃零件1次品")])G_with_costs.add_edges_from([("购买零件2 (C_buyB)", "检验零件2-正品
    (C_checkB)"), ("购买零件2 (C_buyB)", "检验零件2-次品 (C_checkB)"),
    ("购买零件2 (C_buyB)", "不检验零件2"), ("检验零件2-次品 (C_checkB)", "丢弃零件2次品")])

77
78
79 G_with_costs.add_edges_from([("检验零件1-正品 (C_checkA)", "装配成品 (C_assemble)"),

```

```

        ("不检验零件1", "装配成品 (C_assemble)"),
        ("检验零件2-正品 (C_checkB)", "装配成品 (C_assemble)"),
        ("不检验零件2",
        "装配成品 (C_assemble)"))])

80
81
82 G_with_costs.add_edges_from([("装配成品 (C_assemble)", "成品检验-正品 (C_checkC)"),
        ("装配成品 (C_assemble)", "成品检验-次品 (C_checkC)"),
        ("装配成品 (C_assemble)", "不检验成品"))])

83
84
85 G_with_costs.add_edges_from([("成品检验-次品 (C_checkC)", "拆解 (C_disassemble)"),
        ("成品检验-次品 (C_checkC)", "丢弃成品 (L_change)"))])

86
87
88 G_with_costs.add_edges_from([("成品检验-正品 (C_checkC)", "市场检验-合格"),
        ("不检验成品", "市场检验-合格"),
        ("不检验成品", "市场检验-不合格")])

89
90
91 G_with_costs.add_edges_from([("市场检验-不合格", "市场退回 (L_change)"),
        ("市场退回 (L_change)", "拆解 (C_disassemble)"),
        ("市场退回 (L_change)",
        "丢弃成品 (L_change)"))])

92
93
94 G_with_costs.add_edges_from([("市场检验-合格", "流入市场 (S)"))])

95
96
97 G_with_costs.add_edges_from([("拆解 (C_disassemble)", "拆解后零件-重新进入流程"),
        ("拆解后零件-重新进入流程", "检验零件1-正品 (C_checkA)"),
        ("拆解后零件-重新进入流程", "检验零件2-正品 (C_checkB)"))])

98
99
100 pos_with_costs = { "购买零件1 (C_buyA)": (-1, 1), "检验零件1-正品 (C_checkA)": (-1.5, 0.5),
        "检验零件1-次品 (C_checkA)": (-1.5, 0), "不检验零件1": (-0.5, 0.5), "丢弃零件1次品": (-2, 0),
        "购买零件2 (C_buyB)": (1, 1), "检验零件2-正品 (C_checkB)": (1.5, 0.5), "检验零件2-次品
        (C_checkB)": (1.5, 0), "不检验零件2": (0.5, 0.5), "丢弃零件2次品": (2, 0), "装配成品 (C_assemble)":
        (0, 0.5), "成品检验-正品 (C_checkC)": (-0.5, -1), "成品检验-次品 (C_checkC)": (0.5, -1),
        "不检验成品": (0, -0.5), "市场检验-合格": (-0.5, -1.5), "市场检验-不合格": (0.5, -1.5), "流入市场
        (S)": (-1, -2), "丢弃成品 (L_change)": (1, -2), "市场退回 (L_change)": (0, -2), "拆解
        (C_disassemble)": (0, -2.5), "拆解后零件-重新进入流程": (0, -3)}plt.figure(figsize=(14,
        10))nx.draw(G_with_costs, pos=pos_with_costs, with_labels=True, node_size=3000,
        node_color="lightblue", font_size=9, font_weight="bold",
        arrows=True)plt.title("流程图：添加成本注释")plt.show()

```

1 问题二模型求解python代码

2 import itertools

3

4

5 situations = [

6 {'d1': 0.10, 'd2': 0.10, 'd_product': 0.10, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
7 'C_test_product': 3, 'C_market': 56, 'C_exchange': 6, 'C_disassemble': 5, 'quantity': 1000},

8 {'d1': 0.20, 'd2': 0.20, 'd_product': 0.20, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
9 'C_test_product': 3, 'C_market': 56, 'C_exchange': 6, 'C_disassemble': 5, 'quantity': 1000},

```

10 {'d1': 0.10, 'd2': 0.10, 'd_product': 0.10, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
11     'C_test_product': 3, 'C_market': 56, 'C_exchange': 30, 'C_disassemble': 5, 'quantity': 1000},
12 {'d1': 0.20, 'd2': 0.20, 'd_product': 0.20, 'C1': 4, 'C2': 18, 'C_test1': 1, 'C_test2': 1, 'C_assemble': 6,
13     'C_test_product': 2, 'C_market': 56, 'C_exchange': 30, 'C_disassemble': 5, 'quantity': 1000},
14 {'d1': 0.10, 'd2': 0.20, 'd_product': 0.10, 'C1': 4, 'C2': 18, 'C_test1': 8, 'C_test2': 1, 'C_assemble': 6,
15     'C_test_product': 2, 'C_market': 56, 'C_exchange': 10, 'C_disassemble': 5, 'quantity': 1000},
16 {'d1': 0.05, 'd2': 0.05, 'd_product': 0.05, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
17     'C_test_product': 3, 'C_market': 56, 'C_exchange': 10, 'C_disassemble': 40, 'quantity': 1000}
18 ]
19 combinations = list(itertools.product([0, 1], repeat=4))
20
21 best_results = []
22
23 for situation in situations:
24     d1 = situation['d1']
25     d2 = situation['d2']
26     d_product = situation['d_product']
27     C1 = situation['C1']
28     C2 = situation['C2']
29     C_test1 = situation['C_test1']
30     C_test2 = situation['C_test2']
31     C_assemble = situation['C_assemble']
32     C_test_product = situation['C_test_product']
33     C_market = situation['C_market']
34     C_exchange = situation['C_exchange']
35     C_disassemble = situation['C_disassemble']
36     quantity = situation['quantity']
37
38     results = []
39
40     for comb in combinations:
41         x1, x2, x3, x4 = comb
42
43         # 计算成品的次品率 d_total
44         if x1 == 1 and x2 == 1:
45             d_total = d_product
46         elif x1 == 1 and x2 == 0:
47             d_total = (1 - d2) * d_product + d2
48         elif x1 == 0 and x2 == 1:
49             d_total = (1 - d1) * d_product + d1
50         else:
51             d_total = 1 - (1 - d1) * (1 - d2) * (1 - d_product)
52         parts1_quantity = quantity * (1 - x1 * d1)
53         parts2_quantity = quantity * (1 - x2 * d2)
54         assembly_quantity = min(parts1_quantity, parts2_quantity)
55         cost_assemble = C_assemble * assembly_quantity
56         good_products = (1 - d_total) * assembly_quantity
57         revenue = good_products * C_market
58         cost_parts = C1 * quantity + x1 * C_test1 * quantity + C2 * quantity + x2 * C_test2 * quantity
59         cost_product_testing = x3 * C_test_product * assembly_quantity
60         cost_exchange = (1 - x3) * d_total * C_exchange * assembly_quantity
61         P_good1_bad2 = (1 - d1) * d2
62         P_bad1_good2 = d1 * (1 - d2)

```

```

63 P_good1_good2_bad_product = (1 - d1) * (1 - d2) * d_product
64 scrap_value = P_good1_bad2 * C1 + P_bad1_good2 * C2 + P_good1_good2_bad_product * (C1 +
    C2)
65 cost_disassemble = x4 * C_disassemble * assembly_quantity
66 total_cost = cost_parts + cost_assemble + cost_product_testing + cost_exchange + cost_disassemble -
    scrap_value
67 profit = revenue - total_cost
68 results.append((x1, x2, x3, x4, profit, revenue, total_cost, good_products, assembly_quantity))
69 best_result = sorted(results, key=lambda x: x[4], reverse=True)[0]
70 best_results.append(best_result)
71 for i, result in enumerate(best_results):
72 print(f"情况 {i + 1}: 最优策略 (x1, x2, x3, x4) = {result[:4]} 利润 = {result[4]:.2f} 元, 正品成品数量 =
    {result[7]:.0f}")

```

C附录三：问题三代码

```

1 问题三模型求解matlab代码
2 import itertools
3
4
5 situations = [
6 {'d1': 0.10, 'd2': 0.10, 'd_product': 0.10, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
7  'C_test_product': 3, 'C_market': 56, 'C_exchange': 6, 'C_disassemble': 5, 'quantity': 1000},
8 {'d1': 0.20, 'd2': 0.20, 'd_product': 0.20, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
9  'C_test_product': 3, 'C_market': 56, 'C_exchange': 6, 'C_disassemble': 5, 'quantity': 1000},
10 {'d1': 0.10, 'd2': 0.10, 'd_product': 0.10, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
11  'C_test_product': 3, 'C_market': 56, 'C_exchange': 30, 'C_disassemble': 5, 'quantity': 1000},
12 {'d1': 0.20, 'd2': 0.20, 'd_product': 0.20, 'C1': 4, 'C2': 18, 'C_test1': 1, 'C_test2': 1, 'C_assemble': 6,
13  'C_test_product': 2, 'C_market': 56, 'C_exchange': 30, 'C_disassemble': 5, 'quantity': 1000},
14 {'d1': 0.10, 'd2': 0.20, 'd_product': 0.10, 'C1': 4, 'C2': 18, 'C_test1': 8, 'C_test2': 1, 'C_assemble': 6,
15  'C_test_product': 2, 'C_market': 56, 'C_exchange': 10, 'C_disassemble': 5, 'quantity': 1000},
16 {'d1': 0.05, 'd2': 0.05, 'd_product': 0.05, 'C1': 4, 'C2': 18, 'C_test1': 2, 'C_test2': 3, 'C_assemble': 6,
17  'C_test_product': 3, 'C_market': 56, 'C_exchange': 10, 'C_disassemble': 40, 'quantity': 1000}
18 ]
19 combinations = list(itertools.product([0, 1], repeat=4))
20
21 best_results = []
22
23 for situation in situations:
24 d1 = situation['d1']
25 d2 = situation['d2']
26 d_product = situation['d_product']
27 C1 = situation['C1']
28 C2 = situation['C2']
29 C_test1 = situation['C_test1']
30 C_test2 = situation['C_test2']
31 C_assemble = situation['C_assemble']
32 C_test_product = situation['C_test_product']
33 C_market = situation['C_market']
34 C_exchange = situation['C_exchange']

```



```

35 C_disassemble = situation['C_disassemble']
36 quantity = situation['quantity']
37
38 results = []
39
40 for comb in combinations:
41     x1, x2, x3, x4 = comb
42
43     # 计算成品的次品率 d_total
44     if x1 == 1 and x2 == 1:
45         d_total = d_product
46     elif x1 == 1 and x2 == 0:
47         d_total = (1 - d2) * d_product + d2
48     elif x1 == 0 and x2 == 1:
49         d_total = (1 - d1) * d_product + d1
50     else:
51         d_total = 1 - (1 - d1) * (1 - d2) * (1 - d_product)
52     parts1_quantity = quantity * (1 - x1 * d1)
53     parts2_quantity = quantity * (1 - x2 * d2)
54     assembly_quantity = min(parts1_quantity, parts2_quantity)
55     cost_assemble = C_assemble * assembly_quantity
56     good_products = (1 - d_total) * assembly_quantity
57     revenue = good_products * C_market
58     cost_parts = C1 * quantity + x1 * C_test1 * quantity + C2 * quantity + x2 * C_test2 * quantity
59     cost_product_testing = x3 * C_test_product * assembly_quantity
60     cost_exchange = (1 - x3) * d_total * C_exchange * assembly_quantity
61     P_good1_bad2 = (1 - d1) * d2
62     P_bad1_good2 = d1 * (1 - d2)
63     P_good1_good2_bad_product = (1 - d1) * (1 - d2) * d_product
64     scrap_value = P_good1_bad2 * C1 + P_bad1_good2 * C2 + P_good1_good2_bad_product * (C1 +
        C2)
65     cost_disassemble = x4 * C_disassemble * assembly_quantity
66     total_cost = cost_parts + cost_assemble + cost_product_testing + cost_exchange + cost_disassemble -
        scrap_value
67     profit = revenue - total_cost
68     results.append((x1, x2, x3, x4, profit, revenue, total_cost, good_products, assembly_quantity))
69     best_result = sorted(results, key=lambda x: x[4], reverse=True)[0]
70     best_results.append(best_result)
71     for i, result in enumerate(best_results):
72         print(f"情况 {i + 1}: 最优策略 (x1, x2, x3, x4) = {result[:4]} 利润 = {result[4]:.2f} 元, 正品成品数量 =
            {result[7]:.0f}")
73
74 d = [0.10, 0.10, 0.10, 0.10, 0.10, 0.10, 0.10, 0.10];
75 C = [2, 8, 12, 2, 8, 12, 8, 12];
76 C_test = [1, 1, 2, 1, 1, 2, 1, 2];
77 C_assemble = [8, 8, 8];
78 C_test_half = [4, 4, 4];
79 C_disassemble = 6;
80 C_market = 200;
81 C_exchange = 40;
82 d_product = 0.10;
83
84

```

```

85 N = 1000;
86 max_profit = -Inf;
87 best_strategy = [];
88
89 for x1 = 0:1
90 for x2 = 0:1
91 for x3 = 0:1
92 for x4 = 0:1
93 for x5 = 0:1
94 for x6 = 0:1
95 for x7 = 0:1
96 for x8 = 0:1
97 for x_test_half1 = 0:1
98 for x_test_half2 = 0:1
99 for x_test_half3 = 0:1
100 for x_disassemble = 0:1

```

D附录四：问题三模型检验 matlab 代码

```

1 % 初始化不同零件数量的范围
2 n_values = linspace(100, 10000, 3000); % 不同的零件数量从100到10000，均匀分布的3000个点
3
4 %存储不同数量下的策略和利润
5 profits = zeros(1, length(n_values)); % 存储不同 n 下的利润
6 strategies = zeros(length(n_values), 12); % 存储不同 n 下的策略 % 遍历不同零件数量，执行优化
7 for i = 1:length(n_values)
8     n = n_values(i); % 当前零件数量
9     X0 = ones(1, 12); % 初始猜测变量
10    options = optimoptions('fmincon', 'Display', 'off');
11    % 优化
12    [X_opt, fval] = fmincon(@(X) objective_func(X, n), X0, [], [], [], [], zeros(1, 12), ones(1, 12),
        @constraints, options);
13    % 存储结果
14    profits(i) = -fval; % 最大利润
15    strategies(i, :) = X_opt; % 优化策略
16    end
17    % 绘制不同 n 下的利润变化
18    figure;
19    plot(n_values, profits, '-o', 'LineWidth', 2);
20    xlabel('零件数量 n');
21    ylabel('最大利润');
22    title('不同零件数量 n 对最大利润的影响');
23    grid on; % 绘制不同 n 下的策略变化figure;hold on;
24    for j = 1:12
25        plot(n_values, strategies(:, j), '-o', 'LineWidth', 1.5);
26    end
27    xlabel('零件数量 n');
28    ylabel('策略选择 (0.5 表示不检验, 2 表示检验)');
29    title('不同零件数量 n 对策略的影响');
30    legend('X_1 (零件1)', 'X_2 (零件2)', 'X_3 (零件3)', 'X_4 (零件4)', 'X_5 (零件5)', ... 'X_6 (零件6)',

```

```

    'X_7 (零件7)', 'X_8 (零件8)', 'X_9 (半成品1)', 'X_{10} (半成品2)', ...    'X_{11} (半成品3)',
    'X_{12} (成品)', 'Location', 'best');
31 grid on;
32 hold off;
33 % ----- 函数部分放在文件末尾 ----- function profit = objective_func(X, n)
34 % 成本参数
35 C_part = [2, 8, 12, 2, 8, 12, 8, 12]; % 零件采购成本
36 C_part_test = [1, 1, 2, 1, 1, 2, 1, 2]; % 零件检测成本 C_semi_test = [4, 4, 4]; % 半成品检测成本
37 C_final_test = 6; % 成品检测成本
38 C_replace = 40; % 成品退货替换成本
39 C_semi_assembly = 8; % 半成品装配成本 C_final_assembly = 6; % 成品装配成本 C_disassembly = 5;
    % 拆解成本
40 % 丢弃损失
41 C_part_loss = C_part; % 零件丢弃损失
42 C_semi_loss = C_semi_assembly + sum(C_part) / 3; % 半成品丢弃损失
43 C_final_loss = C_final_assembly + sum(C_part) + sum(C_part_test); % 成品丢弃损失
44 % 零件通过检验或不检验后的数量（检验的值设为2，不检验的设为0.5） parts_passed = n * (X(1:8) == 2)
    * 1 + (X(1:8) == 0.5) * 0.5; % 通过检验的零件数量
45 % 计算半成品数量，根据流程图的零件组合
46 N_semi(1) = min([parts_passed(1), parts_passed(2), parts_passed(3)]); % 半成品1
47 N_semi(2) = min([parts_passed(4), parts_passed(5), parts_passed(6)]); % 半成品2
48 N_semi(3) = min([parts_passed(7), parts_passed(8)]); % 半成品3
49 % 通过检验的半成品数量（同样，检验设为2，不检验设为0.5）
50 semi_passed = N_semi .* (X(9:11) == 2) * 1 + (X(9:11) == 0.5) * 0.5; % 半成品的检测结果
51 % 计算成品数量（假设三个半成品组成一个成品）
52 N_final = min(semi_passed); % 成品数量取决于最少的半成品数量
53 % 计算总成本
54 parts_cost = sum(n * (C_part + X(1:8) .* C_part_test)); % 零件成本 + 检测成本 semi_test_cost =
    sum(N_semi .* (X(9:11) .* C_semi_test)); % 半成品检测成本
55 final_test_cost = N_final * X(12) * C_final_test; % 成品检测成本
56 d_f = 1 - X(9) * X(10) * X(11); % 成品次品率 replacement_cost = N_final * (1 - X(12)) * d_f *
    C_replace; % 市场退货替换成本
57 disassembly_cost = N_final * (1 - X(12)) * d_f * C_disassembly; % 拆解成本
58 parts_loss = sum(n * (1 - X(1:8)) .* C_part_loss); % 零件丢弃损失
59 semi_loss = sum(N_semi .* (1 - X(9:11)) .* C_semi_loss); % 半成品丢弃损失
60 final_loss = N_final * (1 - X(12)) * C_final_loss; % 成品丢弃损失
61 assembly_cost = N_final * (3 * C_semi_assembly + C_final_assembly); % 装配成本
62 total_cost = parts_cost + semi_test_cost + final_test_cost + replacement_cost + disassembly_cost
    + assembly_cost + parts_loss + semi_loss + final_loss;
63 % 计算收入
64 revenue = N_final * 200 * (1 - d_f); % 假设每个成品售价为 200 profit = revenue - total_cost; %
    收入减去成本即为利润
65 end
66 % 约束函数 function [c, ceq] = constraints(X) d_f = 1 - X(9) * X(10) * X(11); % 成品次品率
    d_f_max = 0.15; % 成品次品率的最大允许值 c = d_f - d_f_max; % 非线性不等式约束 ceq = []; %
    无等式约束 end
67 % 初始化不同零件数量的范围
68 n_values = linspace(100, 10000, 3000); % 不同的零件数量从100到10000，均匀分布的3000个点
69 % 存储不同数量下的策略和利润
70 profits = zeros(1, length(n_values)); % 存储不同 n 下的利润
71 strategies = zeros(length(n_values), 12); % 存储不同 n 下的策略
72 % 遍历不同零件数量，执行优化
73 for i = 1:length(n_values)

```

```

74 n = n_values(i); % 当前零件数量
75 X0 = ones(1, 12); % 初始猜测变量
76 options = optimoptions('fmincon', 'Display', 'off');
77 % 优化
78 [X_opt, fval] = fmincon(@(X) objective_func(X, n), X0, [], [], [], [], zeros(1, 12), ones(1, 12),
    @constraints, options);
79 % 存储结果
80 profits(i) = -fval; % 最大利润
81 strategies(i, :) = X_opt; % 优化策略
82 end
83 % 绘制不同 n 下的利润变化
84 figure;
85 plot(n_values, profits, '-o', 'LineWidth', 2);
86 xlabel('零件数量 n');
87 ylabel('最大利润');
88 title('不同零件数量 n 对最大利润的影响');
89 grid on;
90 % 绘制不同 n 下的策略变化
91 figure;
92 hold on;
93 for j = 1:12
94     plot(n_values, strategies(:, j), '-o', 'LineWidth', 1.5);
95 end
96 xlabel('零件数量 n');
97 ylabel('策略选择 (0.5 表示不检验, 2 表示检验)');
98 title('不同零件数量 n 对策略的影响');
99 legend('X_1 (零件1)', 'X_2 (零件2)', 'X_3 (零件3)', 'X_4 (零件4)', 'X_5 (零件5)', ... 'X_6
    (零件6)', 'X_7 (零件7)', 'X_8 (零件8)', 'X_9 (半成品1)', 'X_{10} (半成品2)', ... 'X_{11}
    (半成品3)', 'X_{12} (成品)', 'Location', 'best');
100 grid on;
101 hold off;
102 % ----- 函数部分放在文件末尾 -----
103 function profit = objective_func(X, n)
104 % 成本参数
105 C_part = [2, 8, 12, 2, 8, 12, 8, 12]; % 零件采购成本
106 C_part_test = [1, 1, 2, 1, 1, 2, 1, 2]; % 零件检测成本
107 C_semi_test = [4, 4, 4]; % 半成品检测成本 C_final_test = 6; % 成品检测成本
108 C_replace = 40; % 成品退货替换成本 C_semi_assembly = 8; % 半成品装配成本 C_final_assembly = 6;
    % 成品装配成本 C_disassembly = 5; % 拆解成本 % 丢弃损失 C_part_loss = C_part; %
    零件丢弃损失
109 C_semi_loss = C_semi_assembly + sum(C_part) / 3; % 半成品丢弃损失
110 C_final_loss = C_final_assembly + sum(C_part) + sum(C_part_test); % 成品丢弃损失
111 % 零件通过检验或不检验后的数量 (检验的值设为2, 不检验的设为0.5)
112 parts_passed = n * (X(1:8) == 2) * 1 + (X(1:8) == 0.5) * 0.5; % 通过检验的零件数量
113 % 计算半成品数量, 根据流程图的零件组合
114 N_semi(1) = min([parts_passed(1), parts_passed(2), parts_passed(3)]); % 半成品1
115 N_semi(2) = min([parts_passed(4), parts_passed(5), parts_passed(6)]); % 半成品2
116 N_semi(3) = min([parts_passed(7), parts_passed(8)]); % 半成品3
117 %通过检验的半成品数量 (同样, 检验设为2, 不检验设为0.5) semi_passed = N_semi .* ((X(9:11) == 2)
    * 1 + (X(9:11) == 0.5) * 0.5);
118 % 半成品的检测结果
119 % 计算成品数量 (假设三个半成品组成一个成品)
120 N_final = min(semi_passed);

```

```

121 % 成品数量取决于最少的半成品数量 % 计算总成本 parts_cost = sum(n * (C_part + X(1:8) .*
    C_part_test));
122 % 零件成本 + 检测成本
123 semi_test_cost = sum(N_semi .* (X(9:11) .* C_semi_test)); % 半成品检测成本
124 final_test_cost = N_final * X(12) * C_final_test; % 成品检测成本
125 d_f = 1 - X(9) * X(10) * X(11); % 成品次品率
126 replacement_cost = N_final * (1 - X(12)) * d_f * C_replace; % 市场退货替换成本
127 disassembly_cost = N_final * (1 - X(12)) * d_f * C_disassembly; % 拆解成本
128 parts_loss = sum(n * (1 - X(1:8)) .* C_part_loss); % 零件丢弃损失
129 semi_loss = sum(N_semi .* (1 - X(9:11)) .* C_semi_loss); % 半成品丢弃损失 final_loss = N_final * (1
    - X(12)) * C_final_loss; % 成品丢弃损失
130 assembly_cost = N_final * (3 * C_semi_assembly + C_final_assembly); % 装配成本
131 total_cost = parts_cost + semi_test_cost + final_test_cost + replacement_cost + disassembly_cost
    + assembly_cost + parts_loss + semi_loss + final_loss; % 计算收入
132 revenue = N_final * 200 * (1 - d_f); % 假设每个成品售价为 200
133 profit = revenue - total_cost; % 收入减去成本即为利润
134 end
135 % 约束函数 function [c, ceq] = constraints(X) d_f = 1 - X(9) * X(10) * X(11);
136 % 成品次品率
137 d_f_max = 0.15;
138 % 成品次品率的最大允许值
139 c = d_f - d_f_max; % 非线性不等式约束
140 ceq = []; % 无等式约束
141 end

```
